Woxsen University

School of Technology



PROJECT REPORT

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR BUS RESERVATION AND FLEET SCHEDULING SYSTEM

B. Tech. - Artificial Intelligence and Machine Learning

Submitted by:
Harshit Palla
25WU0102095

ACKNOWLEDGMENT

I would like to express my sincere gratitude to all those who havecontributed to the successful completion of this project on the Design and Implementationof a Database Management System for Bus Reservation and Fleet Scheduling System.

First and foremost, I extend my heartfelt thanks to my project guide, [Faculty Name], [Designation], for their guidance, continuous support and constructive feedback during the reviews of this project. Their suggestions helped me move from the first idea to a working database design.

I am grateful to [Head of Department Name], Head of the Department, and to the School of Technology, Woxsen University, for providing the facilities and resources required for this project.

My sincere thanks to my peers and friends who gave useful ideas and feedback during the different phases of the project.

Finally, I am deeply grateful to my family for their constant support and encouragement throughout my academic journey.

Harshit Palla

ABSTRACT

Bus operators manageroutes,stops,buses,seats,schedules,crews, bookings, cancellations, fares and payments.Whenthisworkisdonemanually,thesameseat can be sold twice, a driver can be given two tripsatthesametime,andreportsarehardtoprepare. This project designs and implements a relationaldatabaseforaBusReservationandFleet Scheduling System that prevents these problems.

The system was analysedtofinditsusers,functionalrequirementsand major entities. An entity relationship modelwaspreparedandconvertedintoarelational schema of 13 tables, normalized up toThirdNormalForm(3NF).Thedatabasewasimplemented in MySQL 8 with primary keys, foreignkeys,UNIQUE,NOTNULL,CHECK, ENUM and DEFAULT constraints, six indexesonfrequentlysearchedcolumns,andsixtriggers. A generated column, active_seat, makessurethataseathasonlyoneactivebookingper trip while still allowing a cancelled seat to bebookedagain.

The important businessrulesareenforcedinsidethedatabaseitself: one booking per seat per trip, no overlappingdutiesforadriver,conductororbus,validdeparture and arrival times, payment equal tothefare,andacancellationpolicythatrefunds90%, 50% or 0% of the fare depending on thetimeleftbeforedeparture.Theserulesweretested directly in MySQL, and every test gave theexpectedresult.Atthetimeofwriting,thesample data, report queries and the Python front-endarestillinprogressandaredescribedasremaining work.

This work shows howcarefuldesign,normalizationanddatabase-level rules can remove the main sources of errorinamanualbusbookingprocess.

Keywords: DatabaseManagementSystem,EntityRelationshipModel, Normalization, Third Normal Form, IntegrityConstraints,Triggers,MySQL,BusReservation, Fleet Scheduling

TABLE OF CONTENTS

CERTIFICATE i

DECLARATION of the candidate ii

ACKNOWLEDGMENT iii

ABSTRACT iv

LIST OF TABLES vii

LIST OF FIGURES viii

1. INTRODUCTION 1

1.1 Background 1

1.2 Motivation 1

1.3 Problem Statement 1

1.4 Objectives 2

1.5 Users and Stakeholders 2

1.6 Functional Requirements 3

1.7 Scope and Limitations 3

1.8 Organization of the Report 3

2. TECHNOLOGY REVIEW 5

2.1 Relational Model 5

2.2 Entity Relationship Modelling 5

2.3 Normalization 5

2.4 Integrity Constraints 5

2.5 Triggers 6

2.6 Indexes 6

2.7 Transactions and Concurrency 6

2.8 MySQL and Python 6

2.9 Research Gap 7

3. METHODOLOGY; DESIGN AND IMPLEMENTATION 8

3.1 System Overview 8

3.2 Requirement Analysis 8

3.3 Conceptual Design: ER Diagram 9

3.4 Design Refinement 10

3.5 Relational Schema 11

3.6 Normalization 12

3.7 Data Dictionary 13

3.8 Integrity Constraints..... 17
3.9 Business Rules and Triggers 17
3.10 Indexing 19
3.11 Queries, Views and Reports 19
3.12 Tools and Environment 20
4. RESULTS AND DISCUSSION 21
4.1 Implementation Status 21
4.2 Verification of the Business Rules 21
4.3 Discussion 23
4.4 Comparison with the Manual System 23
5. CONCLUSION AND FUTURE WORK 24
5.1 Summary of Contributions 24
5.2 Limitations 24
5.3 Future Work 24
5.4 Concluding Remarks 25
REFERENCES 26
APPENDIX –I 27
I.1 Rule Test Transcript (MySQL 8.0.46) 27
I.2 Screenshots to Be Added 28
APPENDIX –II 29

LIST OF TABLES

Table 1.1: Users and Stakeholders

Table 1.2: Functional Requirements

Table 2.1: Database Features Used in the Project

Table 3.1: Major Entities Identified

Table 3.2: Design Refinement from Review 1 to the Implemented Schema

Table 3.3: Relational Schema: Primary and Foreign Keys

Table 3.4: Functional Dependencies and Normal Form

Table 3.5: Data Dictionary

Table 3.6: Integrity Constraints Implemented

Table 3.7: Business Rules and How They Are Enforced

Table 3.8: Cancellation Refund Policy

Table 3.9: Indexes

Table 3.10: Planned Reports and Queries

Table 3.11: Tools and Environment

Table 4.1: Implementation Status

Table 4.2: Business Rule Test Results

Table 4.3: Refund Tier Test Results

Table 4.4: Manual System versus Database-Driven System

1. INTRODUCTION

A bus company has many things to keep track of at the same time: routes, stops, buses, seats, schedules, drivers, conductors, passengers, bookings, cancellations, fares and payments. When this information is kept in registers or spreadsheets, mistakes are easy to make. The most serious mistake is selling the same seat to two passengers on the same trip.

This project designs and builds a relational database for a Bus Reservation and Fleet Scheduling System. The database is implemented in MySQL, and the important business rules are enforced by the database itself, so they cannot be skipped by mistake.

1.1 Background

Bus operators run trips on fixed routes. Each trip uses one bus, one driver and one conductor, and passengers reserve individual seats. In many small operations, booking is still done with paper registers, phone calls and spreadsheets. These tools do not check whether a seat is already taken or whether a crew member is already on another trip.

A relational database management system (DBMS) stores data in related tables and can check rules automatically [1]. By converting an entity relationship (ER) model into a normalized schema and adding constraints and triggers, the rules of the business can be written once and enforced for every user and every application.

1.2 Motivation

The motivation for this project comes from the following points:

- **Avoid double booking:** a seat should never be sold twice on the same trip.
- **Fleet visibility:** bus, seat and route data should be in one place so that availability can be checked quickly.
- **Safe crew scheduling:** a driver or conductor should not be assigned to two overlapping trips.
- **Reliable reports:** occupancy, revenue, cancellations and fleet utilization should be available without manual work.
- **Learning outcome:** the project applies ER modelling, normalization, integrity constraints, triggers and indexing in one complete system.

1.3 Problem Statement

Bus operators manageroutes, stops, vehicles, seats, schedules, crews, bookings, cancellations and fares. Manual booking can allocate the same seat twice. The main problems addressed by this project are:

- Seats can be double-booked because there is no system-level check.
- There is no fleet-wide view; bus, seat and route data are scattered and availability cannot be verified in real time.
- Drivers or conductors can be assigned to two overlapping trips without anyone noticing.
- There is no structured reporting; occupancy, revenue, cancellations and utilization cannot be tracked reliably.

The aim is to design and implement a normalized relational database, with suitable constraints, that removes these problems.

1.4 Objectives

Thespecificobjectives of this project are:

1. To model all entities and relationships accurately and convert the ER model into a relational schema.
2. To normalize the schema up to Third Normal Form (3NF) and remove redundancy.
3. To enforce integrity with primary key, foreign key, UNIQUE, NOT NULL, CHECK and DEFAULT constraints.
4. To prevent double booking and crew duty overlap through rules inside the database.
5. To deliver a working front-end that demonstrates the core booking workflows.

1.5 Users and Stakeholders

Sixkindsofpeopleuseordependon the system. Table 1.1 lists them andwhat each one does.

User	Role in the system
Administrator/Operator	Configures routes, buses, seats, schedulesand fares.
Booking Clerk	Manages counter bookings and cancellations for passengers.
Passenger	Searches trips, books or cancels seats, and makes payments.
Driver	Is assigned to trips; subject to duty-time and overlap rules.

User	Role in the system
Conductor	Is assigned to trips; manages onboard ticketing and fare collection.
Finance / Reports Viewer	Reviews revenue, occupancy, cancellation and utilization reports.

Table 1.1: Users and Stakeholders

1.6 Functional Requirements

The system must support theseven functions in Table 1.2.

No.	Function	Description
1	Trip Publishing	Create and publish schedules fora route and bus.
2	Availability Search	Check seat availability by route,date and time.
3	Seat Booking	Reserve a specific seat for a specific trip.
4	Cancellation and Refund	Cancel a booking under the policy and trigger the refund.
5	Crew Assignment	Assign a driver and conductor without duty overlap.
6	Fare Collection	Record and reconcile payments against bookings.
7	Reporting and Analytics	Occupancy, revenue, cancellations, crew schedules, fleet utilization and passenger history.

Table 1.2: Functional Requirements

1.7 Scope and Limitations

The scope of the project is:

- End-to-end management of routes, stops, fleet, seats and schedules.
- Crew handling: driver and conductor assignment and duty tracking.
- Passenger booking, cancellation, fare and payment workflows.
- Operational reports: occupancy, revenue, cancellations and utilization.
- Implementation on MySQL with a Python front-end for the core booking workflows.

The limitations are:

- There is no login or role-based access for administrator, clerk and passenger.

- Fares are set per route and bus type only. There is no seat-level pricing, per-stop fare or dynamic fare for peak days.
- A booking covers a whole trip; booking only a part of the route (from one stop to another) is not supported.
- At the time of writing, sample data, report queries, the front-end and end-to-end testing are still in progress (see Chapter 4).

2. TECHNOLOGY REVIEW

This chapter explains the database concepts and tools that the project is built on.

2.1 Relational Model

The relational model stores data in tables made of rows and columns. A primary key identifies each row, and a foreign key links a row to a row in another table [1]. Data is read and changed using SQL (Structured Query Language). The model is simple, and it has strong support for rules that keep data correct.

2.2 Entity Relationship Modelling

An ER model describes a system using entities (things we store data about), attributes (their properties) and relationships (how entities are connected) [2]. Relationships have a cardinality such as one-to-many (1:N) or many-to-many (M:N). When an ER model is converted into tables, three common rules are used: a 1:N relationship puts a foreign key on the "many" side; an M:N relationship becomes a separate table that holds both keys; and a 1:1 relationship uses a foreign key with a UNIQUE constraint.

2.3 Normalization

Normalization organizes tables so that each fact is stored in only one place. It uses functional dependencies: if the value of A decides the value of B, we write A → B [3], [4]. A table is in First Normal Form (1NF) when every column holds a single value. It is in Second Normal Form (2NF) when no non-key column depends on only a part of the key. It is in Third Normal Form (3NF) when no non-key column depends on another non-key column [6]. Normalized tables avoid repeated data and update mistakes.

2.4 Integrity Constraints

Integrity constraintsare rules that the database checks every time dataisadded or changed.
Table 2.1 lists thefeatures of MySQL used in this project and the purposeof each [5].

Feature	Purpose in this project
PRIMARY KEY	Identifies each row of a table uniquely.
FOREIGN KEY	Makes sure a trip, booking or payment points to a row that really exists.

Feature	Purpose in this project
UNIQUE	Stops duplicates, for example the same busregistration number or the same seat number in a bus.
NOT NULL	Makes sure important values are never left empty.
CHECK	Limits values, for example distance above 0 or arrival after departure.
ENUM	Allows only a fixed list of values, for example bus type or booking status.
DEFAULT	Fills a standard value when none is given, for example status Confirmed.
Generated column	Keeps a value calculated by the database; used for active_seat.
Trigger	Runs automatically before or after a change and can reject it; used for rules that need other rows or tables.
Index	Speeds up searching on a column.

Table 2.1: Database Features Used in the Project

2.5 Triggers

A trigger is a small program stored in the database that runs automatically when a row is inserted, updated or deleted. It can run BEFORE the change, to check or adjust it, or AFTER the change, to update other tables. A trigger can stop a change by raising an error with the SIGNAL statement. A CHECK constraint can only look at the row being saved, so triggers are used for rules that need to look at other rows or tables, such as overlapping trips [5].

2.6 Indexes

An index is a separate structure that lets the database find rows quickly without reading the whole table. MySQL uses B-tree indexes for most columns. Indexes make searches faster but add a small cost to every insert and update, so they are created only on columns that are searched often, such as the source and destination city of a route or the departure time of a trip.

2.7 Transactions and Concurrency

A transaction groups several steps so that they all succeed or all fail (COMMIT or ROLLBACK). The InnoDB storage engine of MySQL supports transactions and keeps data consistent when many users work at the same time [3]. A UNIQUE index is checked by the database itself, so even two clerks booking the same seat at the same moment cannot both

succeed. For steps that need several checks, such as booking and then paying, the application should run them inside one transaction.

2.8 MySQL and Python

MySQL 8 was chosen as the database server. From version 8.0.16 it enforces CHECK constraints, which earlier versions accepted but ignored [5]. The front-end is planned in Python, which can talk to MySQL through a connector library and run SQL statements, transactions and reports.

2.9 Research Gap

Simple booking programs often check their rules only in application code. If another program or a person edits the data directly, those rules are bypassed. This project places the important rules inside the database, so they hold no matter which front-end is used.

3. METHODOLOGY: DESIGN AND IMPLEMENTATION

This chapter describes how the system was designed and built, from requirements to the final MySQL implementation.

3.1 System Overview

The work followed eight steps, shown in Figure 3.1. Steps 1 to 4 are analysis and design, steps 5 and 6 are implementation and verification, and steps 7 and 8 are the reporting and application layer.

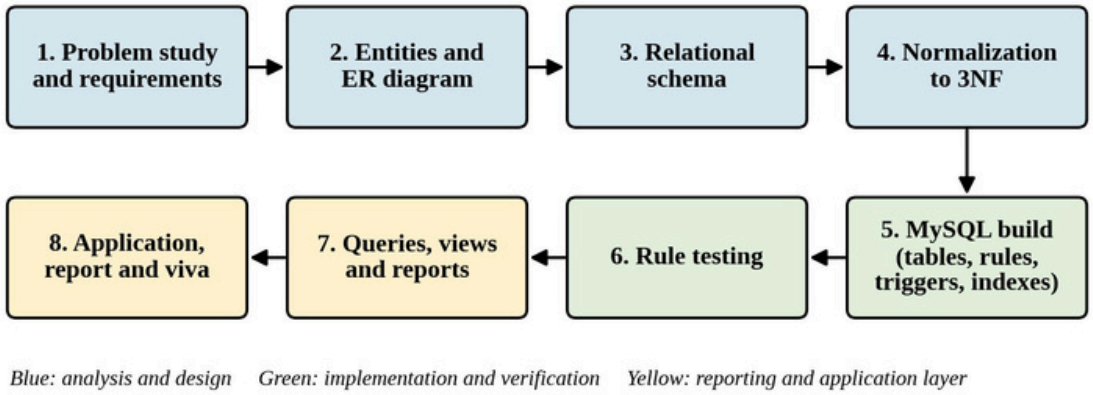


Figure 3.1: Methodology Flowchart

3.2 Requirement Analysis

From the problem description, twelve major entities were identified in Review 1. Table 3.1 lists them.

Entity	Meaning
Route	Source, destination and path definition.
Stop	Intermediate halting points on a route.
Bus	Physical vehicle in the fleet.
Seat	Individual seat within a bus.
Schedule / Trip	A specific run of a bus on a route.
Driver	Licensed staff operating the bus.
Conductor	Onboard staff managing tickets and fares.

Entity	Meaning
Passenger	Customer making bookings.
Booking	Reservation of a seat for a trip.
Cancellation	Record of a cancelled booking.
Fare	Price applicable per route and bus type.
Payment	Amount collected against a booking.

Table 3.1: Major Entities Identified

3.3 Conceptual Design: ER Diagram

Figure 3.2 shows the ER diagram of the final design. Master tables (blue) hold reference data, and transaction tables (green) record what happens each day. The crow's foot symbol marks the "many" side of a relationship, and the bar marks a one-to-one link. The main relationships are:

- One **route** has many **trips**, many **fares** and many stops; stops are linked through **route_stop**, which makes route and stop a many-to-many relationship.
- One **bus** has many **seats** and is used in many **trips**.
- One **driver** and one **conductor** can work on many trips, but each trip has exactly one driver and one conductor.
- One **trip** has many **bookings**; one **seat** and one **passenger** can appear in many bookings.
- One **booking** has at most one **payment** and at most one **cancellation**.

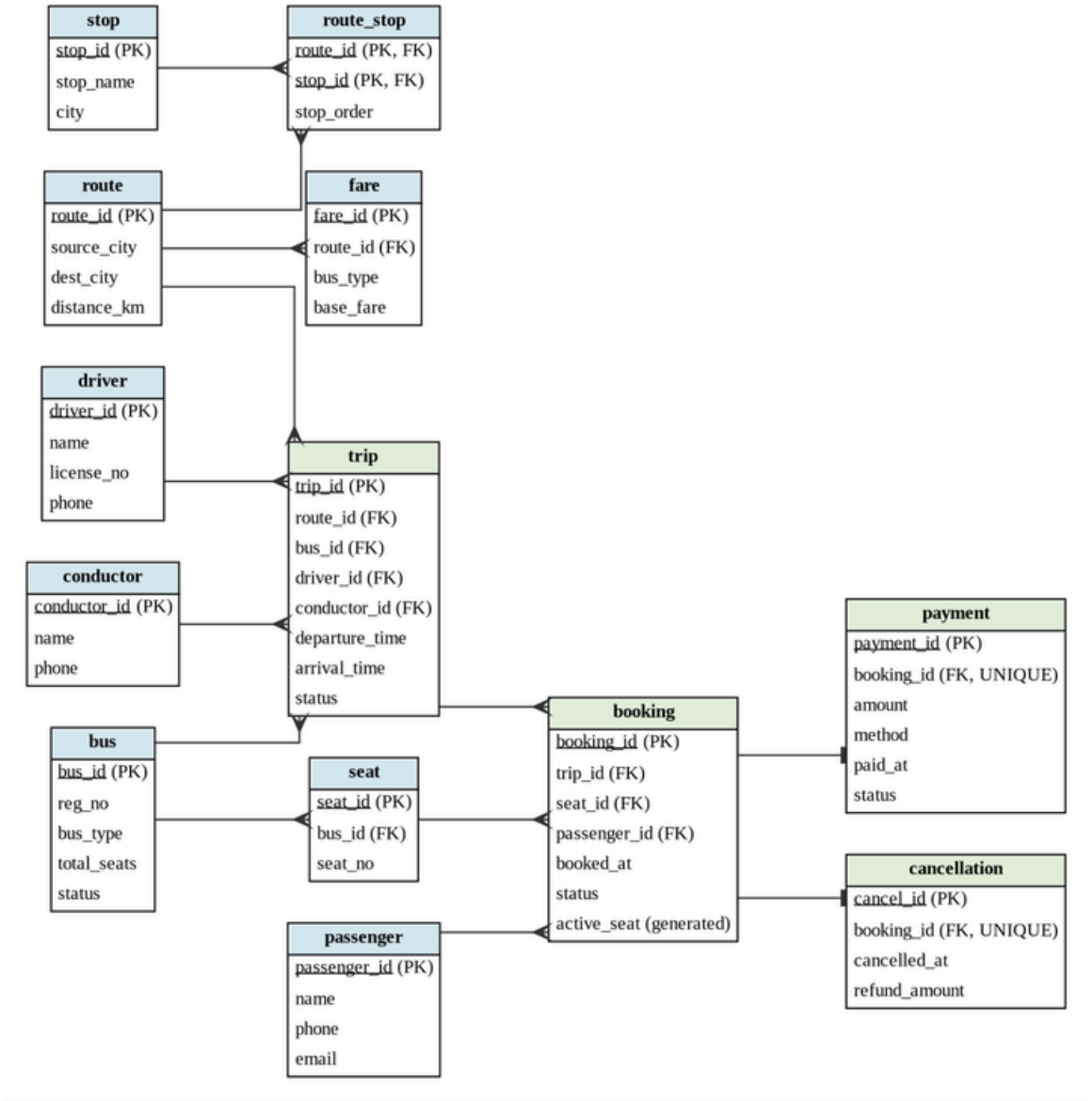


Figure 3.2: Entity Relationship Diagram

3.4 Design Refinement

Thedesignpresented in Review 1 was improved during implementation. Table 3.2 shows what changed. The number of tables stayed at 13: route_stop was added and Crew_Assignment was merged into trip.

Review 1 design	Implemented design
Schedule / Trip (schedule_id)	trip, which now also holds route, bus,driver and conductor.
Crew_Assignment (M:N table)	Removed. driver_id and conductor_id are stored in trip, because a trip has one driver and one conductor.

Review 1 design	Implemented design
Stop (route_id as foreignkey)	stop and route_stop with stop_order,soa stop can be shared by many routes.
UNIQUE (schedule_id, seat_id)	UNIQUE (trip_id, active_seat), so a cancelled seat can be booked again.
Crew overlap "prevented by constraints"	BEFORE INSERT and BEFORE UPDATE triggers on trip for driver, conductor and bus.
Refund "derived from fare and time"	A trigger computes the refund: 90%, 50% or 0% by hours to departure.

Table 3.2: Design Refinement from Review 1 to the Implemented Schema

3.5 Relational Schema

The ER diagram was converted into13tables: 9 master tables and 4 transaction tables. Table 3.3 gives theprimary key (PK) andforeign keys (FK) of each table.

Table	Primary key	Foreign key(s)
Master tables		
route	route_id	—
stop	stop_id	—
route_stop	route_id, stop_id	route_id → route; stop_id → stop
bus	bus_id	—
seat	seat_id	bus_id → bus
driver	driver_id	—
conductor	conductor_id	—
fare	fare_id	route_id → route
passenger	passenger_id	—
Transaction tables		
trip	trip_id	route_id → route; bus_id → bus; driver_id → driver; conductor_id → conductor
booking	booking_id	trip_id → trip; seat_id → seat; passenger_id → passenger

Table	Primary key	Foreign key(s)
payment	payment_id	booking_id → booking (unique)
cancellation	cancel_id	booking_id → booking (unique)

Table 3.3: Relational Schema: Primary and Foreign Keys

The core chain of the system is route → trip → booking → payment or cancellation. A trip also links a bus, a driver and a conductor, and a booking also links a seat and a passenger.

3.6 Normalization

Each table was checked against the three normal forms. Table 3.4 lists the functional dependencies. In every table, the left side of each dependency is a key (primary or candidate), so all tablesare in 3NF.

Table	Functional dependencies	Normal form
route	route_id → source_city, dest_city, distance_km (source_city, dest_city) → route_id	3NF
stop	stop_id → stop_name, city (stop_name, city) → stop_id	3NF
route_stop	(route_id, stop_id) → stop_order	3NF
bus	bus_id → reg_no, bus_type, total_seats, status reg_no → bus_id	3NF
seat	seat_id → bus_id, seat_no (bus_id, seat_no) → seat_id	3NF
driver	driver_id → name, license_no, phone license_no → driver_id	3NF
conductor	conductor_id → name, phone phone → conductor_id	3NF
fare	fare_id → route_id, bus_type, base_fare (route_id, bus_type) → fare_id, base_fare	3NF
passenger	passenger_id → name, phone, email phone → passenger_id	3NF
trip	trip_id → route_id, bus_id, driver_id, conductor_id, departure_time, arrival_time, status	3NF

Table	Functional dependencies	Normal form
booking	booking_id → trip_id, seat_id, passenger_id, booked_at, status	3NF
payment	payment_id → booking_id, amount, method, paid_at, status booking_id → payment_id	3NF
cancellation	cancel_id → booking_id, cancelled_at, refund_amount booking_id → cancel_id	3NF

Table 3.4: Functional Dependencies and Normal Form

1NF: every column holds one value and there are no repeating groups.Atrip does not store a list of seats; each seat is a separate booking row.

2NF: only route_stop has a composite key, and its only non-key column, stop_order, depends on the whole key (route_id, stop_id). All other tables have a single-column key.

3NF: no non-key column depends on another non-key column. For example, a trip stores only the ids of its bus, driver and route. The bus type, driver name and cities are stored once in their own tables and are found by joining.

Three columns need a note. The column active_seat in booking is calculated by the database from seat_id and status; it is not typed in by users, so it cannot become inconsistent, and it exists only to enforce the booking rule. The column refund_amount in cancellation is computed by a trigger and stored as a record of what was refunded. The column total_seats in bus records the declared capacity of the bus; it is not automatically compared with the number of rows in seat (see Section 5.2).

3.7 Data Dictionary

Table 3.5 describes every column of every table.

Column	Type	Constraints	Description
route			
route_id	INT	PK, AUTO_INCREMENT	Unique route number.
source_city	VARCHAR(50)	NOT NULL; UNIQUE with dest_city	Starting city.

Column	Type	Constraints	Description
dest_city	VARCHAR(50)	NOT NULL; CHECK differs from source_city	Destination city.
distance_km	INT	NOT NULL; CHECK > 0	Distance in kilometres.
stop			
stop_id	INT	PK, AUTO_INCREMENT	Unique stop number.
stop_name	VARCHAR(80)	NOT NULL; UNIQUE with city	Name of the stop.
city	VARCHAR(50)	NOT NULL	City of the stop.
route_stop			
route_id	INT	PK, FK → route	Route that contains the stop.
stop_id	INT	PK, FK → stop	Stop on the route.
stop_order	INT	NOT NULL; CHECK > 0; UNIQUE per route	Position of the stop on the route.
bus			
bus_id	INT	PK, AUTO_INCREMENT	Unique bus number.
reg_no	VARCHAR(20)	NOT NULL, UNIQUE	Registration number.
bus_type	ENUM	NOT NULL	Seater, Sleeper, AC Seater or AC Sleeper.
total_seats	INT	NOT NULL; CHECK > 0	Declared seat capacity.
status	ENUM	NOT NULL; DEFAULT Active	Active, Maintenance or Retired.
seat			
seat_id	INT	PK, AUTO_INCREMENT	Unique seat number in the system.
bus_id	INT	NOT NULL, FK → bus	Bus that has the seat.

Column	Type	Constraints	Description
seat_no	VARCHAR(5)	NOT NULL; UNIQUE with bus_id	Seat label such asA1.
driver			
driver_id	INT	PK, AUTO_INCREMENT	Unique driver number.
name	VARCHAR(80)	NOT NULL	Driver name.
license_no	VARCHAR(30)	NOT NULL, UNIQUE	Driving licence number.
phone	VARCHAR(15)	NOT NULL, UNIQUE	Contact number.
conductor			
conductor_id	INT	PK, AUTO_INCREMENT	Unique conductor number.
name	VARCHAR(80)	NOT NULL	Conductor name.
phone	VARCHAR(15)	NOT NULL, UNIQUE	Contact number.
fare			
fare_id	INT	PK, AUTO_INCREMENT	Unique fare number.
route_id	INT	NOT NULL, FK → route	Route the fare applies to.
bus_type	ENUM	NOT NULL; UNIQUE with route_id	Bus type the fare applies to.
base_fare	DECIMAL(8,2)	NOT NULL; CHECK > 0	Ticket price.
passenger			
passenger_id	INT	PK, AUTO_INCREMENT	Unique passenger number.
name	VARCHAR(80)	NOT NULL	Passenger name.
phone	VARCHAR(15)	NOT NULL, UNIQUE	Contact number.
email	VARCHAR(100)	UNIQUE	Email address (optional).

Column	Type	Constraints	Description
trip			
trip_id	INT	PK, AUTO_INCREMENT	Unique trip number.
route_id	INT	NOT NULL, FK → route	Route of the trip.
bus_id	INT	NOT NULL, FK → bus	Bus used.
driver_id	INT	NOT NULL, FK → driver	Driver assigned.
conductor_id	INT	NOT NULL, FK → conductor	Conductor assigned.
departure_time	DATETIME	NOT NULL	Planned departure.
arrival_time	DATETIME	NOT NULL; CHECK > departure_time	Planned arrival.
status	ENUM	NOT NULL; DEFAULT Scheduled	Scheduled, Running, Completed or Cancelled.
booking			
booking_id	INT	PK, AUTO_INCREMENT	Unique booking number.
trip_id	INT	NOT NULL, FK → trip	Trip booked.
seat_id	INT	NOT NULL, FK → seat	Seat booked.
passenger_id	INT	NOT NULL, FK → passenger	Passenger who booked.
booked_at	DATETIME	NOT NULL; DEFAULT current time	Time of booking.
status	ENUM	NOT NULL; DEFAULT Confirmed	Confirmed or Cancelled.
active_seat	INT	Generated (stored); UNIQUE with trip_id	Equals seat_id while Confirmed, otherwise NULL.
payment			

Column	Type	Constraints	Description
payment_id	INT	PK, AUTO_INCREMENT	Unique payment number.
booking_id	INT	NOT NULL, UNIQUE, FK → booking	Booking that was paid.
amount	DECIMAL(8,2)	NOT NULL; CHECK > 0	Amount paid.
method	ENUM	NOT NULL	Cash, Card, UPI or NetBanking.
paid_at	DATETIME	NOT NULL; DEFAULT current time	Time of payment.
status	ENUM	NOT NULL; DEFAULT Paid	Paid or Refunded.
cancellation			
cancel_id	INT	PK, AUTO_INCREMENT	Unique cancellation number.
booking_id	INT	NOT NULL, UNIQUE, FK → booking	Booking that was cancelled.
cancelled_at	DATETIME	NOT NULL; DEFAULT current time	Time of cancellation.
refund_amount	DECIMAL(8,2)	NOT NULL; DEFAULT 0; CHECK ≥ 0	Refund, set by a trigger.

Table 3.5: Data Dictionary

3.8 Integrity Constraints

Table 3.6 summarizes the constraints used in the database.

Constraint	Where it is used
Primary keys	Every table has an INT AUTO_INCREMENT key; route_stop uses the composite key (route_id, stop_id).
Foreign keys (13)	trip → route, bus, driver, conductor; booking → trip, seat, passenger; payment and cancellation → booking; seat → bus; fare → route; route_stop → route and stop.

Constraint	Where it is used
UNIQUE and NOT NULL	reg_no, license_no, phone and email areunique; also route (source, destination), stop (name, city),seat (bus, seat number), fare (route, bus type).
CHECK (8)	distance > 0; source differs from destination; stop_order > 0; total_seats > 0; base_fare > 0; arrival after departure; payment amount > 0; refund ≥ 0.
ENUM	bus_type, bus status, trip status, booking status, payment method, payment status.
DEFAULT	Active, Scheduled, Confirmed, Paid, current time for booking, payment and cancellation times, and 0 for the refund.
Generated column	active_seat in booking.

Table 3.6: Integrity Constraints Implemented

3.9 Business Rules and Triggers

The businessrules of Review 1 are enforced as shown in Table 3.7. Six triggers are used for the rules thataCHECK constraint cannot express.

Rule	How it is enforced	Table
One bookingper seat per trip	UNIQUE (trip_id, active_seat) onagenerated column.	booking
No overlapping crew duties	BEFORE INSERT and BEFORE UPDATE triggers reject a trip if its driver, conductor or bus already has a non-cancelled trip in the same time window.	trip
Valid departure and arrival times	CHECK (arrival_time > departure_time).	trip
Valid seat and open trip	BEFORE INSERT trigger: the seat must belong to the trip's bus and the trip must be Scheduled.	booking
Payment consistency	BEFORE INSERT trigger: the amount must equal the base fare for the trip's route and bus type.	payment
Cancellation policy	BEFORE INSERT trigger: blocks repeat or post-departure cancellations and computes the refund.	cancellation

Rule	How it is enforced	Table
Automatic clean-up	AFTER INSERT trigger: marks thebooking Cancelled (which frees the seat) andthe payment Refunded.	cancellation
One payment and one cancellation per booking	UNIQUE (booking_id).	payment, cancellation

Table 3.7: Business Rules and How They Are Enforced

3.9.1 One Active Booking per Seat per Trip

A simple UNIQUE (trip_id, seat_id) constraint would stop double booking, but it would also stop a cancelled seat from being sold again, because the cancelled row would still exist. To solve this, the booking table has a generated column active_seat, which equals seat_id while the booking is Confirmed and is NULL when it is Cancelled. The constraint UNIQUE (trip_id, active_seat) ignores NULL values, so a seat can have many cancelled bookings but only one confirmed booking on a trip.

3.9.2 Cancellation Policy

The cancellation trigger finds the departure time of the trip and the amount paid, and sets the refund according to Table 3.8. A cancellation after departure is rejected.

Time before departure	Refund of the amount paid
24 hours or more	90%
6 hours to less than 24 hours	50%
Less than 6 hours	0%
After departure	Cancellation notallowed

Table 3.8: Cancellation Refund Policy

3.9.3 Payment Consistency

When a payment is inserted, the trigger looks up the base fare for the route and bus type of the booked trip. If the amount is different from that fare, the payment is rejected. Each booking can have only one payment record.

3.9.4 No Overlapping Crew Duties

Two time periods overlap when the first starts before the second ends and the first ends after the second starts. The trip triggers use this test for the driver, the conductor and the bus, and ignore trips that are cancelled. The same test runs on INSERT and on UPDATE, so a trip cannot be moved into a clash later.

3.10 Indexing

Indexes were created on the columns that are searched most often. Table 3.9 lists them. The UNIQUE and foreign key constraints also create indexes automatically.

Index	Table and columns	Speeds up
idx_route_cities	route (source_city, dest_city)	Searching routes bycity pair.
idx_trip_route_dt	trip (route_id, departure_time)	Finding trips of a route on a date.
idx_trip_driver	trip (driver_id, departure_time)	Driver schedules and the overlap check.
idx_trip_conductor	trip (conductor_id, departure_time)	Conductor schedules and the overlap check.
idx_trip_bus	trip (bus_id, departure_time)	Fleet utilization and the bus overlap check.
idx_booking_pass	booking (passenger_id)	Passenger booking history.

Table 3.9: Indexes

3.11 Queries, Views and Reports

The project requires reports built with joins, aggregate functions, nested queries and views. Table 3.10 shows the planned reports. These are part of the work in progress described in Chapter 4.

Report	Tables used	SQL features
Trip occupancy	trip, booking, bus	JOIN, COUNT, GROUPBY, percentage of seats booked
Route revenue	route, trip, booking, payment	JOIN, SUM, GROUP BY route
Cancellations	cancellation, booking, trip, route	JOIN, COUNT, SUM of refunds

Report	Tables used	SQL features
Crew schedules	trip, driver, conductor, route	JOIN, ORDER BY departure time, view
Fleet utilization	bus, trip	LEFT JOIN, COUNT of trips and hours per bus
Passenger history	passenger, booking, trip, route, payment	JOIN, subquery, view

Table 3.10: Planned Reports and Queries

3.12 Tools and Environment

Item	Details
Database	MySQL 8.0.16 or later (rule tests were run on MySQL 8.0.46).
Database script	Single SQL script that creates the bus_system database, tables, indexes and triggers (Appendix II).
Front-end	Python application connected to MySQL through a connector library (in progress).
Diagrams	Graphviz for the ER diagram; Python (Matplotlib) for the flowchart and architecture figure.
Version control	Git and GitHub for the project repository.

Table 3.11: Tools and Environment

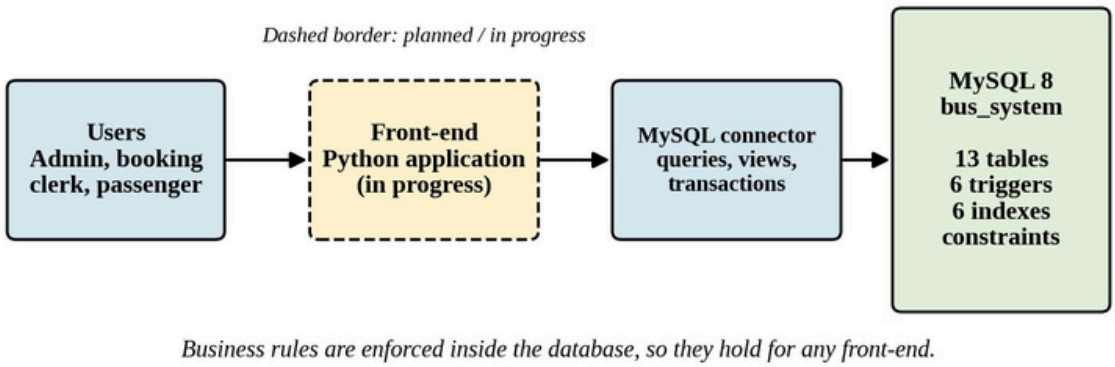


Figure 3.3: System Architecture

Figure 3.3 shows the layers of the system. The MySQL database is built and enforces the rules. The Python front-end is in progress and sits on top of the same database.

4. RESULTS AND DISCUSSION

This chapter reports what has been built so far and the results of testing the business rules.

4.1 Implementation Status

Table 4.1 shows thestatus of each module at the time of writing.

Module	What it does	Status
Schema, constraints, indexes	13 tables with PK, FK, UNIQUE, CHECK, ENUM and indexes	Completed
Triggers	Overlap, booking, payment and cancellation rules	Completed
Sample data	Routes, buses, seats, crews, trips and bookings	In progress
Queries, views, reports	Joins, aggregates, occupancy and revenue views	In progress
Application: search and book	Search trips, pick a seat, pay	In progress
Application: cancel and refund	Cancel a booking, show the refund tier	In progress
Application: admin	Routes, buses, crew and fares	In progress
End-to-end testing	All business rules through theapplication	In progress

Table 4.1: Implementation Status

4.2 Verification of the Business Rules

The schema script was run on MySQL 8.0.46. Each rule was then tested directly in MySQL by inserting a small set of rows and trying valid and invalid actions. Table 4.2 gives each test and the response of the database. The error messages shown are the ones returned by MySQL. The full transcript is in Appendix I.

No.	Action	Result
1	Insert a trip, book a seat and pay the correct fare	Accepted
2	Book the same seat again on the same trip	Rejected: ERROR 1062, duplicate entry for uq_one_booking_per_seat

No.	Action	Result
3	Insert a trip whose driverisalready on an overlapping trip	Rejected: Driver already has an overlapping trip
4	Insert a trip whose conductor is already on an overlapping trip	Rejected: Conductor already has an overlapping trip
5	Insert a trip whose bus is already on an overlapping trip	Rejected: Bus already has an overlapping trip
6	Move an existing trip so that its driver or conductor overlaps (UPDATE)	Rejected: overlapping trip message
7	Insert a trip with arrival before departure	Rejected: Check constraint chk_trip_times is violated
8	Book a seat that belongs to a different bus	Rejected: Seat does not belong to the bus of this trip
9	Book a seat on a trip that is not Scheduled	Rejected: Trip is not open for booking
10	Pay an amount different from the fare (100 instead of 650)	Rejected: Payment amount does not match the fare
11	Cancel a booking after the trip has departed	Rejected: Cannot cancel after departure
12	Cancel the same booking twice	Rejected: Booking is already cancelled
13	Book a seat again after its booking was cancelled	Accepted: active_seat was NULL

Table 4.2: Business Rule Test Results

The refund policy was tested with three trips at different times before departure. Each booking was paid at the fare of 650. Table 4.3 shows the refund calculated by the trigger. In every case the booking became Cancelled and the payment became Refunded.

Time before departure (test trip)	Expected refund	Refund recorded
About 3 days (24 hours or more)	90% of 650 = 585.00	585.00
About 9 hours (6 to 24 hours)	50% of 650 = 325.00	325.00
About 2 hours (under 6 hours)	0% of 650 = 0.00	0.00

Table 4.3: Refund Tier Test Results

These tests were run directly in MySQL. Testing the same rules through the application is still to be done.

4.3 Discussion

Rules inside the database. Every rejection in Table 4.2 came from a constraint or trigger, not from application code. This means the rules hold for any front-end and for anyone who connects to the database directly.

The active_seat column. The generated column solved a problem that a plain UNIQUE constraint could not: it blocks a second confirmed booking for a seat, yet lets the seat be sold again after a cancellation (test 13). Because the check is a UNIQUE index, it also works when two bookings are attempted at the same moment.

Triggers where CHECK is not enough. A CHECK constraint sees only its own row. Overlapping duties, matching the payment to the fare, and the refund tiers all need data from other rows or tables, so triggers were used for them.

A MySQL restriction found during testing. While testing, an INSERT into cancellation that selected its booking ids from the booking table failed with error 1442, because the cancellation trigger updates booking in the same statement. Cancellations must therefore be inserted with explicit booking ids, one booking at a time. The application will do this naturally.

4.4 Comparison with the Manual System

Aspect	Manual system	Database-driven system
Same seat onatrip	Can be sold twice.	Second booking is rejected.
Crew duties	Overlaps go unnoticed.	Overlapping trip is rejected.
Availability	Checked by hand.	Can be found with a query.
Refunds	Decided case by case.	Computed by the 90% / 50%/0% policy.
Payments	Amount may differ from fare.	Must equal the fare.
Reports	Built manually, if at all.	Built from queries and views (in progress).

Table 4.4: Manual System versus Database-Driven System

5. CONCLUSION AND FUTURE WORK

5.1 Summary of Contributions

Thekeycontributions of this projectare:

1. Analysis of the bus reservation and fleet scheduling problem, with users, functional requirements and twelve major entities.
2. An ER model and a refined relational schema of 13 tables, normalized to 3NF, with a data dictionary and functional dependencies.
3. A MySQL implementation with 13 foreign keys, 8 CHECK constraints, ENUM domains, default values, a generated column, 6 indexes and 6 triggers.
4. Database-level enforcement of the main business rules: one active booking per seat per trip, no overlapping duties, valid times, payment equal to fare, and a tiered cancellation refund.
5. Direct testing of these rules in MySQL, with every test giving the expected result.

5.2 Limitations

- Sample data, report queries and views, the Python front-end and end-to-end testing are still in progress.
- A booking can exist without a payment; no rule forces a confirmed booking to be paid.
- The column total_seats in bus is not automatically compared with the number of seat rows of that bus.
- Only overlap is checked for crew duties; minimum rest time between trips is not enforced.
- The overlap triggers read other rows, so under heavy simultaneous use the application should run trip creation inside a transaction. The seat rule does not have this issue because it uses a UNIQUE index.
- There is no login or role-based access, and fares are only per route and bus type.

5.3 Future Work

5.3.1 Complete the Application Layer

Build the Python front-end for trip publishing, availability search, seat booking, cancellation, crew assignment, trip status, fare collection and reporting, using transactions for booking and payment.

5.3.2 Sample Data and Reports

Load realistic sample data and write the joins, aggregates, nested queries and views listed in Table 3.10.

5.3.3 Login and Roles

Add login and role-based access for administrator, booking clerk and passenger.

5.3.4 Better Pricing and Booking

Add seat-level pricing, per-stop fares and dynamic fares for peak days, and allow booking from one stop to another instead of the whole route.

5.3.5 Stronger Rules

Add a rule that links a confirmed booking to its payment, a check of total_seats against the seat list, and a minimum rest time between crew trips.

5.4 Concluding Remarks

This project shows that a well-designed, normalized database with constraints and triggers can remove the main problems of manual bus booking: double-booked seats, clashing crew duties, inconsistent payments and unclear refunds. The database layer is complete and tested, and the remaining work is to add data, reports and the front-end on top of it.

REFERENCES

[1] E. F. Codd, "A relational model of data for large shared data banks," Commun. ACM, vol. 13, no. 6, pp. 377–387, Jun. 1970.

[2] P. P. Chen, "The entity-relationship model—toward a unified view of data," ACM Trans. Database Syst., vol. 1, no. 1, pp. 9–36, Mar. 1976.

[3] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[4] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Hoboken, NJ, USA: Pearson, 2016.

[5] Oracle Corporation, "MySQL 8.0 Reference Manual." [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[6] E. F. Codd, "Further normalization of the data base relational model," in Data Base Systems, R. Rustin, Ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1972, pp. 33–64.

APPENDIX – I
Screen shots of work done

I.1 Rule Test Transcript (MySQL 8.0.46)

The statementsbelow wererunin themysqlclient on the bus_system database. The responses are exactly as returned by MySQL. Long statements are shortened and marked with comments.

```
-- Same seat booked twice on one trip
INSERT INTO booking(trip_id,seat_id,passenger_id) VALUES(1,1,2);
ERROR 1062 (23000): Duplicate entry '1-1' for key 'booking.uq_one_booking_per_seat'

-- Driver, bus and conductor already on an overlapping trip
INSERT INTO trip(...) -- same driver, overlapping time
ERROR 1644 (45000): Driver already has an overlapping trip
INSERT INTO trip(...) -- same bus, overlapping time
ERROR 1644 (45000): Bus already has an overlapping trip
INSERT INTO trip(...) -- same conductor, overlapping time
ERROR 1644 (45000): Conductor already has an overlapping trip
UPDATE trip SET conductor_id=1, departure_time=..., arrival_time=...
WHERE ...;
ERROR 1644 (45000): Conductor already has an overlapping trip

-- Arrival before departure
INSERT INTO trip(...) VALUES(..., '2030-01-02 10:00', '2030-01-02 09:00');
ERROR 3819 (HY000): Check constraint 'chk_trip_times' is violated.

-- Seat from a different bus
INSERT INTO booking(trip_id,seat_id,passenger_id) VALUES(1,4,2);
ERROR 1644 (45000): Seat does not belong to the bus of this trip

-- Booking on a trip that is not Scheduled
UPDATE trip SET status='Completed' WHERE ...;
INSERT INTO booking(...) -- seat of that trip
ERROR 1644 (45000): Trip is not open for booking

-- Wrong payment amount (fare is 650)
INSERT INTO payment(booking_id,amount,method) VALUES(2,100,'UPI');
ERROR 1644 (45000): Payment amount does not match the fare

-- Cancel a booking made for a trip 24 hours or more away
INSERT INTO cancellation(booking_id) VALUES(1);




booking_id status      active_seat
1           Cancelled NULL
payment status: Refunded
refund_amount: 585.00

-- Cancel trips 6-24 hours and under 6 hours away
INSERT INTO cancellation(booking_id) VALUES(1);   -- refund_amount: 325.00
INSERT INTO cancellation(booking_id) VALUES(2);   -- refund_amount: 0.00

-- Book the freed seat again
INSERT INTO booking(trip_id,seat_id,passenger_id) VALUES(1,1,2);
booking_id seat_id status       active_seat
1           1        Cancelled NULL
```

```
4           1        Confirmed 1

-- Cancel twice, and cancel after departure
INSERT INTO cancellation(booking_id) VALUES(1);
ERROR 1644 (45000): Booking is already cancelled
INSERT INTO cancellation(booking_id) VALUES(<booking of departed trip>);
ERROR 1644 (45000): Cannot cancel after departure
```

APPENDIX – II

Database Script (01_schema.sql)

Complete scriptthatcreatesthebus_system database with its tables, constraints, indexes and triggers.

```
-- ========================================================
-- Bus Reservation and Fleet Scheduling System
-- File 1: Schema (tables, constraints, indexes, triggers)
-- Database: MySQL 8.0.16+ (CHECK constraints need 8.0.16+)
-- ========================================================

DROP DATABASE IF EXISTS bus_system;
CREATE DATABASE bus_system;
USE bus_system;

-- ---------- MASTER TABLES ----------

CREATE TABLE route (




    route_id     INT AUTO_INCREMENT PRIMARY KEY,
      source_city VARCHAR(50) NOT NULL,
    dest_city    VARCHAR(50) NOT NULL,
    distance_km INT NOT NULL,
    CONSTRAINT chk_route_cities   CHECK (source_city <> dest_city),
    CONSTRAINT chk_route_distance CHECK (distance_km > 0),
    CONSTRAINT uq_route UNIQUE (source_city, dest_city)
);

CREATE TABLE stop (
    stop_id    INT AUTO_INCREMENT PRIMARY KEY,
      stop_name VARCHAR(80) NOT NULL,
    city       VARCHAR(50) NOT NULL,
    CONSTRAINT uq_stop UNIQUE (stop_name, city)
);

CREATE TABLE route_stop (
    route_id    INT  NOT  NULL,
    stop_id     INT NOT NULL,
    stop_order INT NOT NULL,
    PRIMARY KEY (route_id, stop_id),
    CONSTRAINT uq_route_order UNIQUE (route_id, stop_order),
    CONSTRAINT chk_stop_order CHECK (stop_order > 0),
    FOREIGN KEY (route_id) REFERENCES route(route_id),
    FOREIGN KEY (stop_id) REFERENCES stop(stop_id)
);

CREATE TABLE bus (
    bus_id       INT AUTO_INCREMENT PRIMARY KEY,
    reg_no       VARCHAR(20) NOT NULL UNIQUE,
    bus_type     ENUM('Seater','Sleeper','AC Seater','AC Sleeper') NOT
NULL,
    total_seats INT NOT NULL,
    status       ENUM('Active','Maintenance','Retired') NOT NULL DEFAULT
'Active',
    CONSTRAINT chk_bus_seats CHECK (total_seats > 0)
);
```

```
CREATE TABLE seat (
    seat_id INT AUTO_INCREMENT PRIMARY KEY,
    bus_id   INT NOT NULL,
    seat_no VARCHAR(5)NOTNULL,
    CONSTRAINTuq_seatUNIQUE(bus_id,seat_no),
    FOREIGNKEY(bus_id)REFERENCESbus(bus_id)
);

CREATE TABLE driver (
    driver_id   INTAUTO_INCREMENTPRIMARYKEY,
    name        VARCHAR(80)NOTNULL,
    license_no VARCHAR(30) NOT NULL UNIQUE,
    phone       VARCHAR(15)NOTNULLUNIQUE
);

CREATE TABLE conductor (
    conductor_id  INTAUTO_INCREMENTPRIMARYKEY,
    name          VARCHAR(80)NOTNULL,
    phone         VARCHAR(15)NOTNULLUNIQUE
);

CREATE TABLE fare (
    fare_id    INTAUTO_INCREMENTPRIMARYKEY,
    route_id   INTNOTNULL,
    bus_type   ENUM('Seater','Sleeper','ACSeater','AC Sleeper') NOT NULL,
    base_fare DECIMAL(8,2) NOT NULL,
    CONSTRAINT chk_fare CHECK (base_fare > 0),
    CONSTRAINTuq_fareUNIQUE(route_id,bus_type),
    FOREIGNKEY(route_id)REFERENCESroute(route_id)
);

CREATE TABLE passenger (
    passenger_id  INTAUTO_INCREMENTPRIMARYKEY,
    name   phone  VARCHAR(80)NOTNULL,
    email         VARCHAR(15)NOTNULLUNIQUE,
                  VARCHAR(100)UNIQUE
);

-- ---------- TRANSACTION TABLES ----------

CREATE TABLE trip (
    trip_id         INTAUTO_INCREMENTPRIMARYKEY,
    route_id        INTNOTNULL,
    bus_id          INTNOTNULL,
    driver_id       INTNOTNULL,
    conductor_id    INTNOTNULL,
    departure_time DATETIMENOTNULL,
    arrival_time    DATETIMENOTNULL,
    status          ENUM('Scheduled','Running','Completed','Cancelled')
                    NOTNULLDEFAULT'Scheduled',
    CONSTRAINTchk_trip_timesCHECK(arrival_time > departure_time),
    FOREIGNKEY(route_id)       REFERENCESroute(route_id),
    FOREIGNKEY(bus_id)         REFERENCESbus(bus_id),
    FOREIGNKEY(driver_id)      REFERENCESdriver(driver_id),
    FOREIGNKEY(conductor_id)REFERENCESconductor(conductor_id)
);

-- Rule: one ACTIVE booking per seat per trip.
```

```
-- active_seat is seat_id only while status = 'Confirmed', else NULL.
-- UNIQUE ignores NULLs, so a cancelled seat can be booked again.
CREATE TABLE booking (
    booking_id    INT  AUTO_INCREMENT  PRIMARY  KEY,
    trip_id       INT NOT NULL,
    seat_id       INT NOT NULL,
    passenger_id INT NOT NULL,
    booked_at     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status        ENUM('Confirmed','Cancelled') NOT NULL DEFAULT
'Confirmed',
    active_seat   INT GENERATED ALWAYS AS
                  (IF(status = 'Confirmed', seat_id, NULL)) STORED,
    CONSTRAINT uq_one_booking_per_seat UNIQUE (trip_id, active_seat),
    FOREIGN  KEY  (trip_id)     REFERENCES  trip(trip_id),
    FOREIGN KEY (seat_id)       REFERENCES seat(seat_id),
    FOREIGN KEY (passenger_id) REFERENCES passenger(passenger_id)
);

CREATE TABLE payment (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    booking_id INT NOT NULL UNIQUE,
    amount      DECIMAL(8,2) NOT NULL,
    method      ENUM('Cash','Card','UPI','NetBanking') NOT NULL,
    paid_at     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status      ENUM('Paid','Refunded') NOT NULL DEFAULT 'Paid',
    CONSTRAINT chk_payment_amount CHECK (amount > 0),
    FOREIGN KEY (booking_id) REFERENCES booking(booking_id)
);

CREATE TABLE cancellation (
    cancel_id      INT AUTO_INCREMENT PRIMARY KEY,
    booking_id     INT NOT NULL UNIQUE,
    cancelled_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    refund_amount DECIMAL(8,2) NOT NULL DEFAULT 0,
    CONSTRAINT chk_refund CHECK (refund_amount >= 0),
    FOREIGN KEY (booking_id) REFERENCES booking(booking_id)
);

-- ---------- INDEXES (frequently searched columns) ----------
CREATE INDEX idx_route_cities ON route(source_city, dest_city);
CREATE INDEX idx_trip_route_dt ON trip(route_id, departure_time);

CREATE INDEX idx_trip_driver   ON trip(driver_id, departure_time);
CREATE INDEX idx_trip_conductor ON trip(conductor_id, departure_time);
CREATE INDEX idx_trip_bus      ON trip(bus_id, departure_time);
CREATE INDEX idx_booking_pass ON booking(passenger_id);

-- ---------- TRIGGERS (business rules CHECK cannot handle) ----------
DELIMITER $$

-- Rule: no overlapping duties for driver, conductor or bus
CREATE TRIGGER trg_trip_overlap_ins BEFORE INSERT ON trip
FOR EACH ROW
BEGIN

    IF EXISTS (SELECT 1 FROM trip
               WHERE driver_id = NEW.driver_id AND status <> 'Cancelled'
                 AND NEW.departure_time < arrival_time
                 AND NEW.arrival_time > departure_time) THEN
```

```
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Driver already has an
overlapping trip';
    END IF;
    IF EXISTS (SELECT 1 FROM trip
               WHERE conductor_id = NEW.conductor_id AND status <>
'Cancelled'
                 AND NEW.departure_time < arrival_time
                 AND NEW.arrival_time > departure_time) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Conductor already has
an overlapping trip';
    END IF;
    IF EXISTS (SELECT 1 FROM trip
               WHERE bus_id = NEW.bus_id AND status <> 'Cancelled'
                 AND NEW.departure_time < arrival_time
                 AND NEW.arrival_time > departure_time) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Bus already has an
overlapping trip';
    END IF;
END$$

CREATE TRIGGER trg_trip_overlap_upd BEFORE UPDATE ON trip
FOR EACH ROW
BEGIN
     IF NEW.status <> 'Cancelled' THEN
        IF EXISTS (SELECT 1 FROM trip
                   WHERE driver_id = NEW.driver_id AND trip_id <>
NEW.trip_id
                     AND status <> 'Cancelled'
                     AND NEW.departure_time < arrival_time
                     AND NEW.arrival_time > departure_time) THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Driver already has
an overlapping trip';
        END IF;
        IF EXISTS (SELECT 1 FROM trip
                   WHERE conductor_id = NEW.conductor_id AND trip_id <>
NEW.trip_id
                     AND status <> 'Cancelled'
                     AND NEW.departure_time < arrival_time
                     AND NEW.arrival_time > departure_time) THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Conductor already
has an overlapping trip';
        END IF;
        IF EXISTS (SELECT 1 FROM trip
                   WHERE bus_id = NEW.bus_id AND trip_id <> NEW.trip_id
                     AND status <> 'Cancelled'
                     AND NEW.departure_time < arrival_time
                     AND NEW.arrival_time > departure_time) THEN
            SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Bus already has an
overlapping trip';
        END IF;
    END IF;
END$$

-- Rule: seat must belong to the trip's bus, and trip must still be open
CREATE TRIGGER trg_booking_check BEFORE INSERT ON booking
FOR EACH ROW
BEGIN
    DECLARE v_bus INT;
    DECLARE v_seat_bus INT;
    DECLARE v_status VARCHAR(20);
```

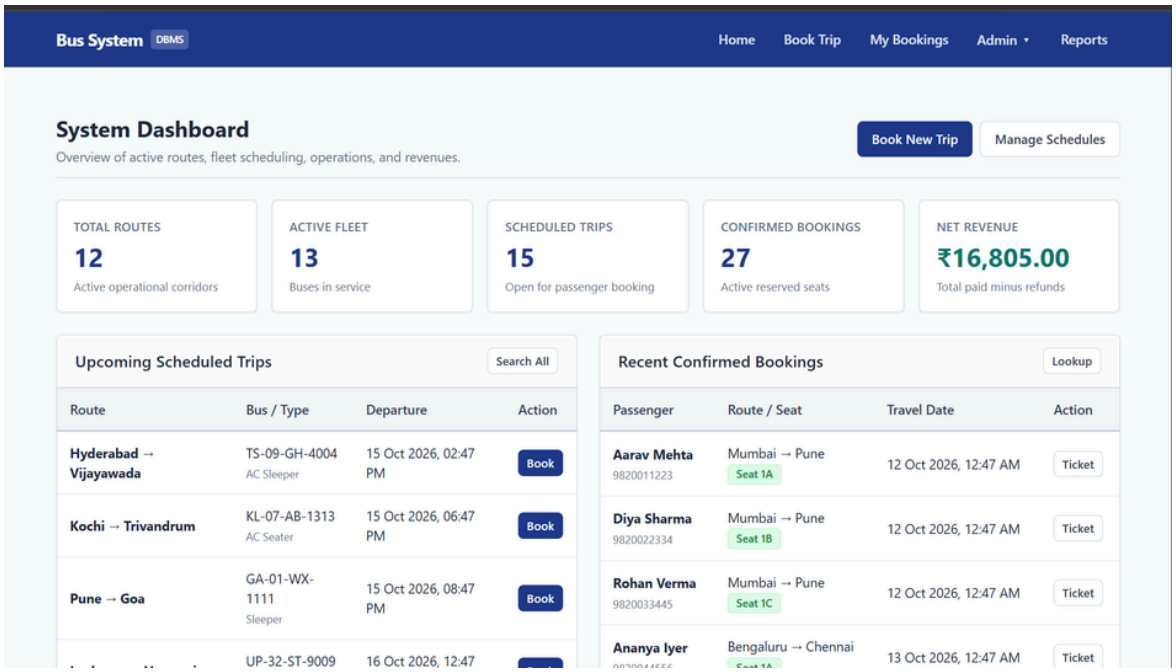
```
    SELECT bus_id, status INTO v_bus, v_status FROM trip WHERE trip_id =
NEW.trip_id;
    SELECT bus_id INTO v_seat_bus FROM seat WHERE seat_id = NEW.seat_id;
    IF v_bus <> v_seat_bus THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Seat does not belong to
the bus of this trip';
    END IF;
    IF v_status <> 'Scheduled' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Trip is not open for
booking';
    END IF;
END$$

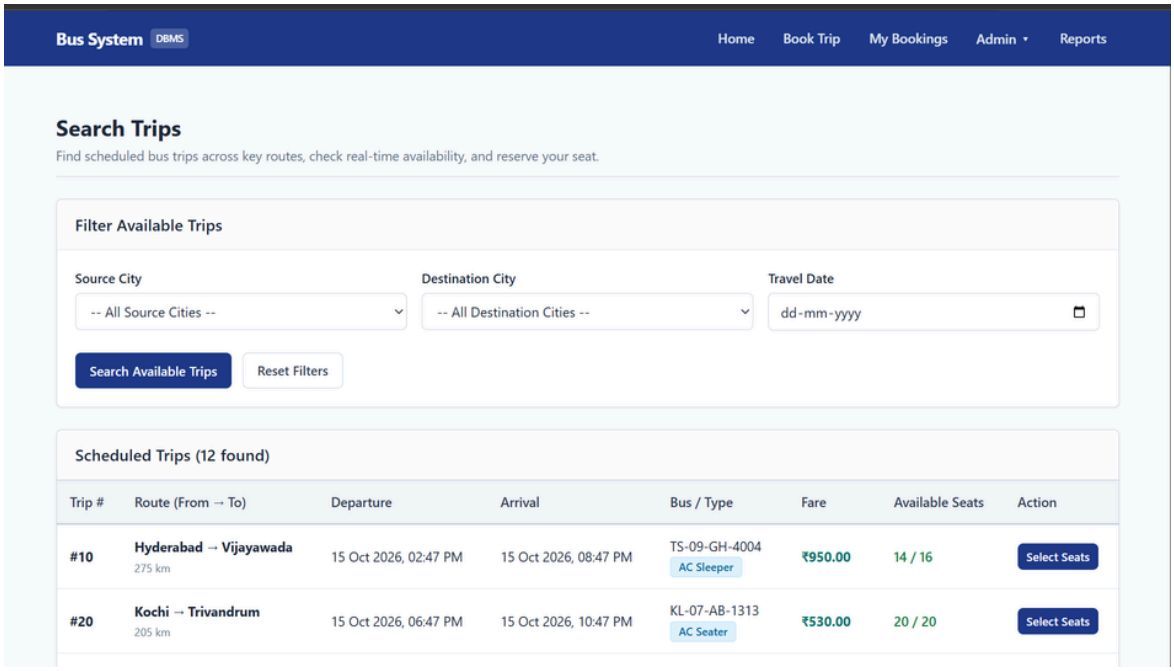
-- Rule: payment amount must equal the fare of that trip
CREATE TRIGGER trg_payment_check BEFORE INSERT ON payment
FOR EACH ROW
BEGIN
    DECLARE v_fare DECIMAL(8,2);
    SELECT f.base_fare INTO v_fare
    FROM booking b
    JOIN trip t ON t.trip_id = b.trip_id
    JOIN bus u ON u.bus_id = t.bus_id
    JOIN fare f ON f.route_id = t.route_id AND f.bus_type = u.bus_type
    WHERE b.booking_id = NEW.booking_id;
    IF v_fare IS NULL OR NEW.amount <> v_fare THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Payment amount does not
match the fare';
    END IF;
END$$

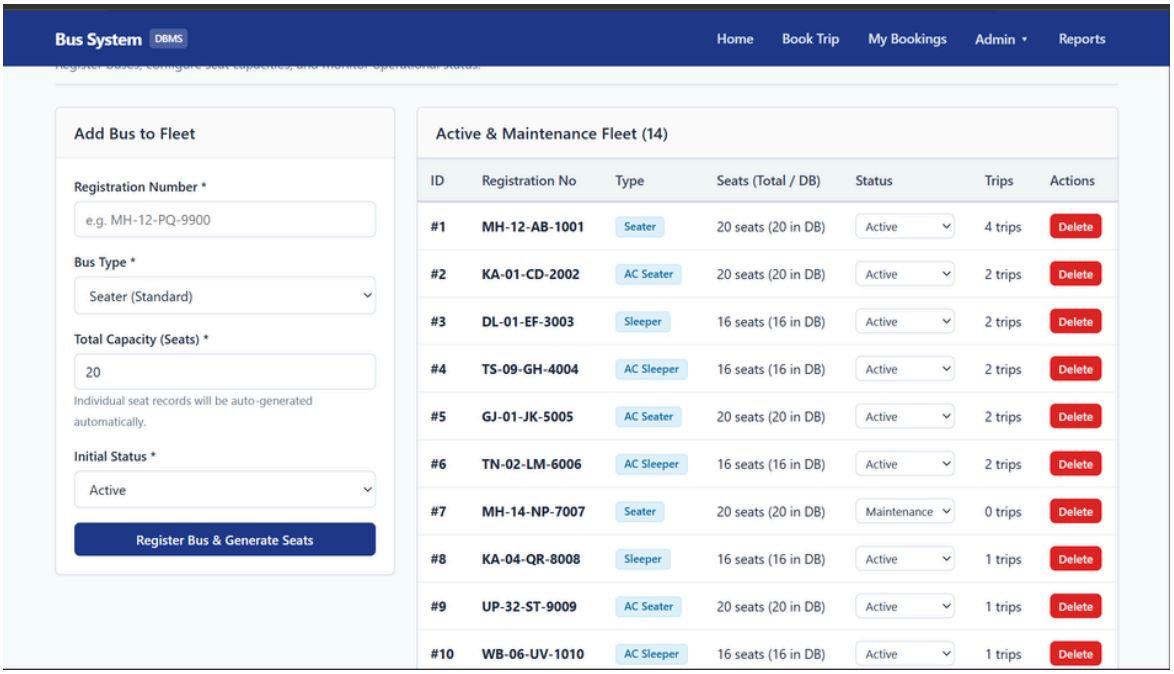
-- Rule: cancellation policy
--   24 hours or more before departure -> 90% refund
--   6 to 24 hours before departure -> 50% refund
--   less than 6 hours -> no refund
--   after departure -> not allowed
CREATE TRIGGER trg_cancel_policy BEFORE INSERT ON cancellation
FOR EACH ROW
BEGIN
    DECLARE v_dep DATETIME;
    DECLARE v_paid DECIMAL(8,2);
    DECLARE v_status VARCHAR(20);
    DECLARE v_hours INT;
    SELECT t.departure_time, b.status INTO v_dep, v_status
    FROM booking b JOIN trip t ON t.trip_id = b.trip_id
    WHERE b.booking_id = NEW.booking_id;
    IF v_status <> 'Confirmed' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Booking is already
cancelled';
END IF;
IF v_dep <= NEW.cancelled_at THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Cannot cancel after
departure';
END IF;
SELECT COALESCE(SUM(amount), 0) INTO v_paid
FROM payment WHERE booking_id = NEW.booking_id;
SET v_hours = TIMESTAMPDIFF(HOUR, NEW.cancelled_at, v_dep);
SET NEW.refund_amount = v_paid * CASE
        WHEN v_hours >= 24 THEN 0.90
        WHEN v_hours >= 6 THEN 0.50
        ELSE 0 END;
END$$
```

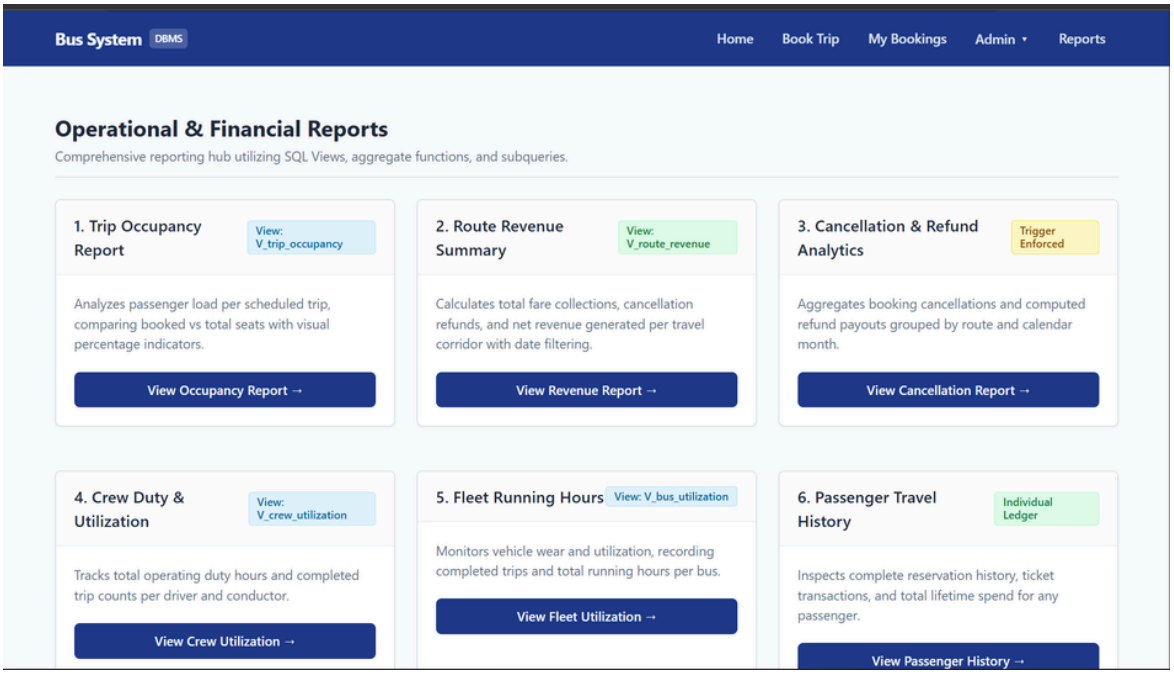
```
--After cancelling: free the seat and mark payment refunded
CREATE TRIGGER trg_cancel_after AFTER INSERT ON cancellation
FOREACH ROW
BEGIN
    UPDATE booking SET status = 'Cancelled' WHERE booking_id=
NEW.booking_id;
    UPDATE payment SET status = 'Refunded' WHERE booking_id=
NEW.booking_id;
END$$

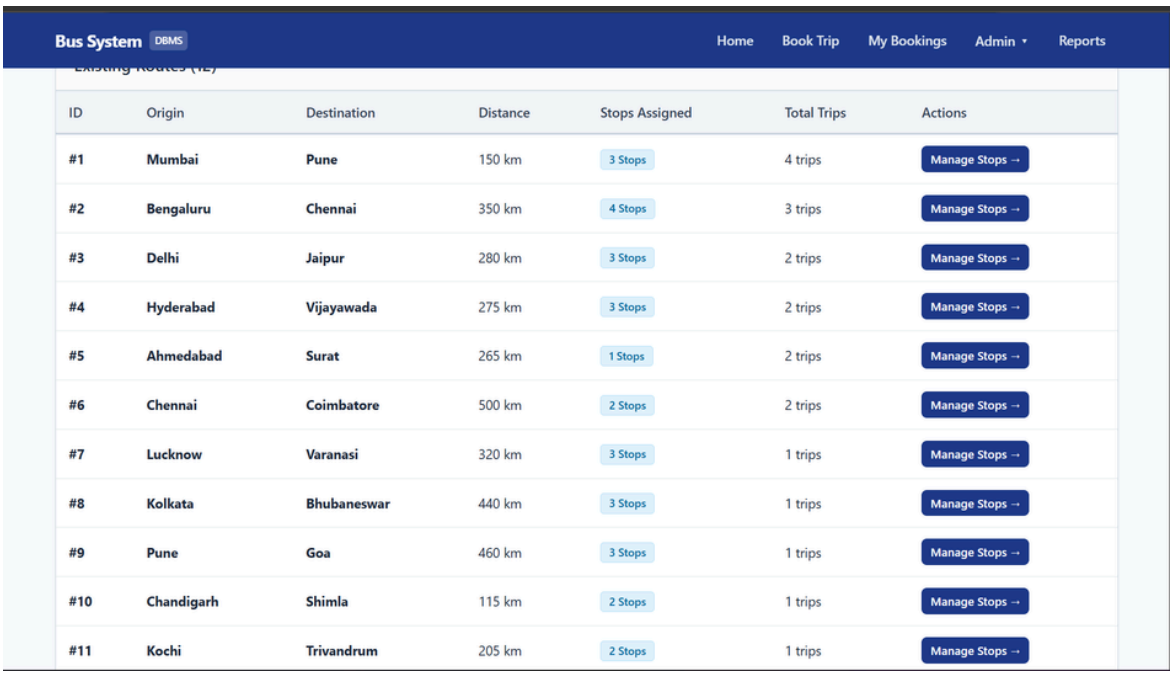
DELIMITER ;
```






Bus System DBMS
Home
Book Trip
My Bookings
Admin
Reports
Add Bus to Fleet
Registration Number *
e.g. MH-12-PQ-9900
Bus Type *
Seater (Standard)
Total Capacity (Seats) *
20
Individual seat records will be auto-generated automatically.
Initial Status *
Active
Register Bus & Generate Seats
Active & Maintenance Fleet (14)
ID
Registration No
Type
Seats (Total / DB)
Status
Trips
Actions
#1 MH-12-AB-1001 Seater 20 seats (20 in DB) Active 4 trips Delete
#2 KA-01-CD-2002 AC Seater 20 seats (20 in DB) Active 2 trips Delete
#3 DL-01-EF-3003 Sleeper 16 seats (16 in DB) Active 2 trips Delete
#4 TS-09-GH-4004 AC Sleeper 16 seats (16 in DB) Active 2 trips Delete
#5 GJ-01-JK-5005 AC Seater 20 seats (20 in DB) Active 2 trips Delete
#6 TN-02-LM-6006 AC Sleeper 16 seats (16 in DB) Active 2 trips Delete
#7 MH-14-NP-7007 Seater 20 seats (20 in DB) Maintenance 0 trips Delete
#8 KA-04-QR-8008 Sleeper 16 seats (16 in DB) Active 1 trips Delete
#9 UP-32-ST-9009 AC Seater 20 seats (20 in DB) Active 1 trips Delete
#10 WB-06-UV-1010 AC Sleeper 16 seats (16 in DB) Active 1 trips Delete


Bus System DBMS
Home
Book Trip
My Bookings
Admin
Reports
Operational & Financial Reports
Comprehensive reporting hub utilizing SQL Views, aggregate functions, and subqueries.
1. Trip Occupancy Report
View: V_trip_occupancy
Analyzes passenger load per scheduled trip, comparing booked vs total seats with visual percentage indicators.
View Occupancy Report →
2. Route Revenue Summary
View: V_route_revenue
Calculates total fare collections, cancellation refunds, and net revenue generated per travel corridor with date filtering.
View Revenue Report →
3. Cancellation & Refund Analytics
Trigger Enforced
Aggregates booking cancellations and computed refund payouts grouped by route and calendar month.
View Cancellation Report →
4. Crew Duty & Utilization
View: V_crew_utilization
Tracks total operating duty hours and completed trip counts per driver and conductor.
View Crew Utilization →
5. Fleet Running Hours
View: V_bus_utilization
Monitors vehicle wear and utilization, recording completed trips and total running hours per bus.
View Fleet Utilization →
6. Passenger Travel History
Individual Ledger
Inspects complete reservation history, ticket transactions, and total lifetime spend for any passenger.
View Passenger History →


Bus System DBMS
Home
Book Trip
My Bookings
Admin
Reports
ID
Origin
Destination
Distance
Stops Assigned
Total Trips
Actions
#1 Mumbai Pune 150 km 3 Stops 4 trips Manage Stops →
#2 Bengaluru Chennai 350 km 4 Stops 3 trips Manage Stops →
#3 Delhi Jaipur 280 km 3 Stops 2 trips Manage Stops →
#4 Hyderabad Vijayawada 275 km 3 Stops 2 trips Manage Stops →
#5 Ahmedabad Surat 265 km 1 Stops 2 trips Manage Stops →
#6 Chennai Coimbatore 500 km 2 Stops 2 trips Manage Stops →
#7 Lucknow Varanasi 320 km 3 Stops 1 trips Manage Stops →
#8 Kolkata Bhubaneswar 440 km 3 Stops 1 trips Manage Stops →
#9 Pune Goa 460 km 3 Stops 1 trips Manage Stops →
#10 Chandigarh Shimla 115 km 2 Stops 1 trips Manage Stops →
#11 Kochi Trivandrum 205 km 2 Stops 1 trips Manage Stops →